Linux Security AI - Sistema Centralizzato di Monitoraggio e Sicurezza

📚 Progetto Didattico per Studenti di Cybersecurity

Sistema completo di monitoraggio, rilevamento malware, code scanning e intrusion detection basato su AI e database vettoriale per analisi semantica.

🎯 Obiettivi Formativi

- Comprendere architetture di sicurezza centralizzate
- Applicare Machine Learning alla cybersecurity
- Implementare anomaly detection e behavioral analysis
- Utilizzare database vettoriali per ricerca semantica
- Gestire privacy, encryption e RBAC in sistemi di sicurezza

🏗️ Architettura del Sistema

```
┌──────────────────────────────────────────────────────────────┐
│                     SERVER LINUX MONITORATI                  │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐    │
│  │Performance│  │ Malware  │  │   Code   │  │   IDS    │    │
│  │  Monitor  │  │ Detector │  │ Scanner  │  │ (Network)│    │
│  └──────────┘  └──────────┘  └──────────┘  └──────────┘    │
└──────────────────────────────────────────────────────────────┘
          │              │              │              │
          │              │              │              │
          │        HTTPS/TLS + Auth Token              │
          └──────────────┴──────────────┴──────────────┘
                         ▼              ▼
          ┌──────────────────────────────────────┐
          │      SERVER CENTRALE (API)           │
          │  ┌──────────────┐  ┌──────────────┐  │
          │  │   FastAPI    │  │  PostgreSQL  │  │
          │  │   Backend    │  │ + SQLAlchemy │  │
          │  └──────────────┘  └──────────────┘  │
          │         │                  │         │
          │         │                  │         │
          │         └──┌──────────────┐┘         │
          │            │ Chroma/Qdrant DB │      │
          │            │  (Vector Search) │      │
          │            └──────────────┘          │
          └──────────────────────────────────────┘
                              │
                              ▼
          ┌──────────────────────────────────────┐
          │    DASHBOARD WEB (HTML/JS)           │
```

```
            │  • Metriche real-time              │
            │  • Lista alert                     │
            │  • Ricerca semantica               │
            │  • Investigation View              │
            └────────────────────────────────────┘
```

🔧 Moduli Implementati

1 Performance Monitor

- Monitoraggio CPU, RAM, Disco, I/O rete
- **Anomaly Detection** con Isolation Forest
- Suggerimenti ottimizzazione
- Privacy: dati aggregati, no cmdline sensibili

2 Malware & Process Behavior Detector

- Analisi statica: hash, entropia, YARA rules
- Analisi comportamentale: network, file ops, privilege escalation
- Classificatore ML (Random Forest)
- Privacy: path tokenizzati, no contenuti file

3 Code Scanner

- Integrazione bandit/semgrep
- Detection: command injection, SQL injection, hardcoded secrets
- Snippet salvati in DB vettoriale per similarità
- Privacy: codice sanitizzato

4 Intrusion Detection System (IDS)

- Log-based: brute force SSH, login anomali
- Network-based: port scan, pattern TCP
- ML-based anomaly detection
- Privacy: IP pseudoanonimizzati, username masked

5 Server Centrale (API + DB)

- FastAPI con endpoints RESTful
- PostgreSQL per dati strutturati
- Autenticazione token/HMAC
- HTTPS/TLS obbligatorio

6 Database Vettoriale (Chroma)

- Embedding con sentence-transformers
- Collezioni: alerts, code_snippets, knowledge

- Ricerca semantica top-k
- Integrazione con API backend

7️⃣ Dashboard Web

- Bootstrap + Chart.js
- Viste: Overview, Alerts, Code Issues
- Ricerca "incidenti simili"
- RBAC: admin, security, viewer

🔒 Sicurezza e Privacy (Design by Privacy)

☑️ Implementato

1. **Data Minimization**: solo dati essenziali raccolti
2. **Sanitizzazione**: rimozione password/token/secrets da tutti i payload
3. **Anonimizzazione**: hashing file/processi/IP
4. **Encryption in Transit**: HTTPS/TLS + HMAC
5. **Encryption at Rest**: Fernet per campi sensibili DB
6. **Retention Policy**: auto-delete dati > 90 giorni
7. **RBAC**: 3 ruoli (admin, security, viewer)
8. **Audit Logging**: log accessi dati sensibili
9. **Validation Pre-Send**: blocco invio dati sensibili

🚀 Quick Start

Prerequisiti

- Docker & Docker Compose
- Linux (Ubuntu 20.04/22.04 consigliato)
- Python 3.9+

Installazione

```
# 1. Clone repository
git clone <repo-url>
cd linux-security-ai

# 2. Copia configurazione template
cp config/config.template.yml config/config.yml
# Modifica config.yml con i tuoi parametri

# 3. Genera certificati self-signed per HTTPS (ambiente didattico)
./scripts/generate_certs.sh

# 4. Build e avvio con Docker Compose
docker-compose up --build
```

```
# 5. Accesso dashboard
# https://localhost:8443
# Username: admin
# Password: changeme (vedi config.yml)
```

Setup Agents su Server Monitorati

```
# Su ogni server Linux da monitorare
cd agents/

# Installa dipendenze
pip install -r requirements.txt

# Configura endpoint server centrale
export CENTRAL_SERVER_URL="https://central-server:8000"
export API_TOKEN="<your-token>"

# Avvia agent (esempio: performance monitor)
python performance/monitor.py
```

📦 Stack Tecnologico

Component	Technology
Backend	FastAPI, Python 3.9+
Database	PostgreSQL + SQLAlchemy
Vector DB	Chroma (embeddings)
ML/AI	scikit-learn, sentence-transformers
Monitoring	psutil, scapy, watchdog
Security	cryptography (Fernet), HMAC
Frontend	HTML5, Bootstrap 5, Chart.js
Deployment	Docker, Docker Compose
Code Analysis	bandit, semgrep, YARA

📖 Documentazione

- Architettura Dettagliata
- Guida Installazione Completa
- Privacy & Security Policy
- API Reference

- Troubleshooting

Esercizi per Studenti

- Esercizio 1: Setup Ambiente
- Esercizio 2: Performance Monitoring
- Esercizio 3: Malware Detection
- Esercizio 4: Vector Search
- Esercizio 5: Incident Response

🧪 Testing

```
# Unit tests
pytest tests/unit/

# Integration tests
pytest tests/integration/

# Coverage report
pytest --cov=agents --cov=central_server tests/
```

📁 Struttura Progetto

```
linux-security-ai/
├── agents/                     # Agent per monitoraggio host
│   ├── common/                 # Utilities comuni (sanitizer, crypto, sender)
│   ├── performance/            # Performance monitor + anomaly detection
│   ├── malware/                # Malware & process behavior detector
│   ├── code_scanner/           # Code scanner (bandit/semgrep)
│   └── ids/                    # Intrusion Detection System
├── central_server/             # Server centrale
│   ├── api/                    # FastAPI endpoints
│   ├── db/                     # SQLAlchemy models
│   ├── vector_search/          # Chroma/Qdrant integration
│   └── security/               # Auth, RBAC, encryption, audit
├── dashboard/                  # Frontend web
│   ├── static/                 # CSS, JS, immagini
│   └── templates/              # HTML templates
├── config/                     # File configurazione
├── docs/                       # Documentazione
│   ├── architecture/
│   ├── guides/
│   └── exercises/
├── docker/                     # Dockerfile per ogni componente
├── scripts/                    # Script utility
├── tests/                      # Test suite
```

```
├── docker-compose.yml
└── README.md
```

🎓 Obiettivi di Apprendimento per Settimana

Settimana 1-2: Setup & Performance Monitoring

- Installazione ambiente Docker
- Implementazione agent performance
- Anomaly detection con Isolation Forest
- Privacy by design basics

Settimana 3: Malware Detection

- Static analysis (hash, entropy, YARA)
- Behavioral analysis
- ML classifier training

Settimana 4: Code Scanning & IDS

- Integrazione bandit/semgrep
- Log-based IDS
- Network-based IDS

Settimana 5: Server Centrale & Vector DB

- API REST con FastAPI
- PostgreSQL + SQLAlchemy
- Chroma vector database
- Sentence transformers

Settimana 6: Dashboard & Security

- Frontend con Bootstrap
- Semantic search integration
- RBAC implementation
- Encryption & audit logging

Settimana 7-8: Testing & Incident Response

- Test suite completa
- Simulazione incidenti
- Investigazione con vector search
- Presentazione progetto

🤝 Contribuire (per Studenti)

1. Fork del repository
2. Crea branch feature (`git checkout -b feature/miglioramento`)
3. Commit modifiche (`git commit -am 'Aggiunto feature X'`)
4. Push al branch (`git push origin feature/miglioramento`)
5. Apri Pull Request

⚠ Disclaimer

Questo è un progetto DIDATTICO.

- ❌ NON usare in ambienti di produzione senza revisione di sicurezza professionale
- ❌ NON eseguire scanning su sistemi non autorizzati
- ☑ Usare solo su VM/container isolati
- ☑ Rispettare leggi e regolamenti sulla privacy (GDPR)
- ☑ Fini educativi e di ricerca

📜 Licenza

MIT License - Progetto didattico per scopi educativi

👥 Autori & Docenti

- **Prof. [Nome Docente]** - Corso di Cybersecurity
- Università/Istituto: [Nome]
- Anno Accademico: 2025/2026

📧 Supporto

Per domande o problemi:

- Issue Tracker GitHub
- Email: [email-docente]
- Forum studenti: [link]

Buon hacking (etico)! 🌓 🔐